

SLR Project P4 Report

Distributed MapReduce and Remote Experiment Infrastructure

Guilherme Caporali [fill later: add teammate names if needed]

June 19, 2026

Contents

1 Introduction

This project comprises two distributed workflows executed over the Telecom Paris laboratory machines: a custom MapReduce framework written in Go and a deployment/collection pipeline used to launch programs remotely and gather machine statistics. The principal engineering contribution is the MapReduce system: the client builds a job-specific worker binary, deploys it to a pool of remote hosts, distributes the input, drives map and reduce over HTTP, and merges the final output locally.

The design objective was to remain compatible with the constraints of a student laboratory environment: no root access, no Docker, shared machines, SSH-based access only, and frequent host churn. The implementation therefore relies on direct use of `/tmp`, SSH/SCP, HTTP, and bounded parallel fan-out rather than on a more elaborate deployment stack.

2 Overall architecture

2.1 System overview

The repository contains two related but distinct distributed systems:

1. **The MapReduce pipeline** exposed by `mapreduce.sh` and implemented mainly in `scripts/mapreduce`, `scripts/worker`, and `scripts/jobs/*`.
2. **The remote stats pipeline** exposed by `run.sh` and implemented in `scripts/make_hosts`, `scripts/deploy`, `scripts/collect`, and `scripts/cleanup`.

Both workflows share the same discovery model: the local machine first queries <https://tp.telcom-paris.fr/ajax.php> to generate `hosts.txt`, then fans out over SSH or HTTP to the selected nodes.

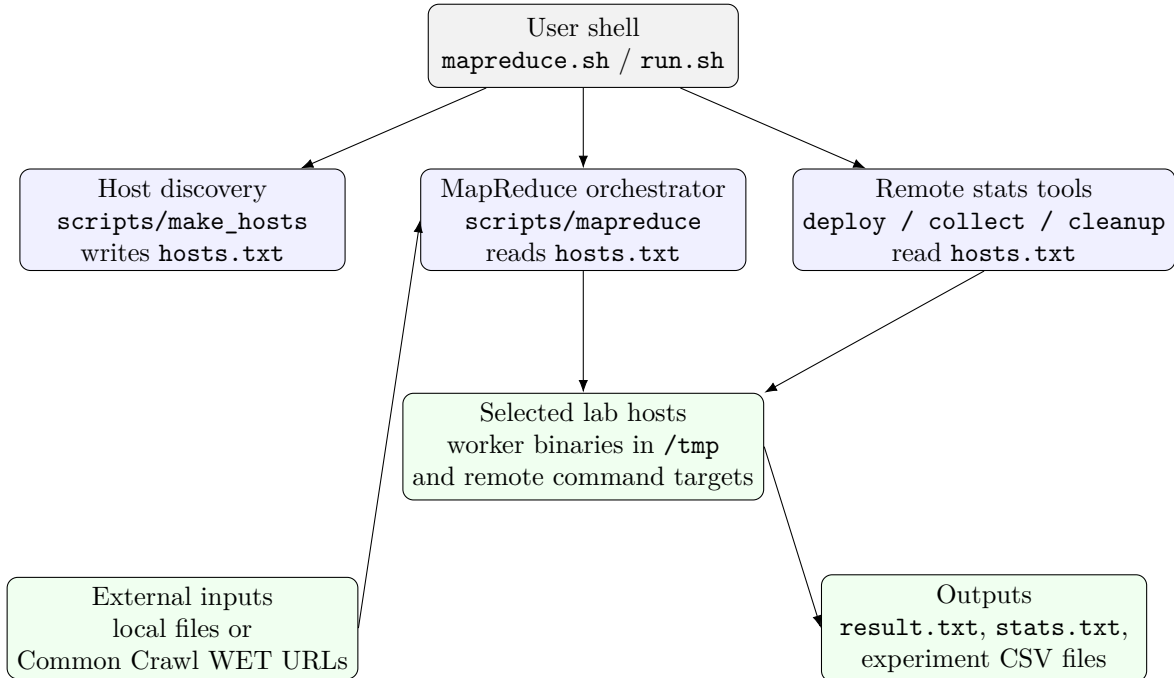


Figure 1: Repository-level architecture. The MapReduce orchestrator and the stats pipeline share the same host-discovery mechanism (`hosts.txt`) but execute different remote workflows on the lab machine pool.

2.2 Main components

Component	Location	Role
Wrapper scripts	<code>mapreduce.sh</code> , <code>run.sh</code>	User-facing entry points. They parse shell flags, fetch hosts, and launch the correct Go commands.
Host discovery	<code>scripts/make_hosts</code>	Downloads the list of available lab machines and writes the first N hosts to <code>hosts.txt</code> .
MapReduce orchestrator	<code>scripts/mapreduce</code>	Builds the worker binary for the selected job, deploys it, assigns input, coordinates phases, handles failures, and merges final results.
Worker server	<code>scripts/worker</code>	Runs one HTTP server per host, stores local input and intermediate files, executes map and reduce, serves shuffle data, and exposes final results.
Job API	<code>scripts/mrjob</code>	Defines the <code>Mapper</code> and <code>Reducer</code> interfaces used by all jobs.
Jobs	<code>scripts/jobs/*</code>	Application logic. The repository already includes <code>wordcount</code> , <code>langdetect</code> , <code>domainpop</code> , and <code>dockdensity</code> .
Remote execution helpers	<code>scripts/internal/remote</code>	Provides bounded parallelism and shared SSH/SCP settings such as connection reuse and timeouts.
Stats pipeline	<code>scripts/deploy</code> , <code>collect</code> , <code>cleanup</code>	Starts remote programs on many nodes, collects command outputs, and performs cleanup.
Kafka comparison setup	<code>kafka/*</code>	Separate benchmark path used to compare the custom system with Kafka Streams on the same machine pool.

2.3 Detailed architecture of the MapReduce system

The MapReduce architecture follows a client-orchestrated model rather than a master that stays permanently deployed in the cluster.

1. The local machine reads `hosts.txt` and decides how many hosts are active slots and how many are hot spares.
2. The orchestrator cross-compiles a Linux worker binary that already contains the chosen job package. This is important: the remote worker does not dynamically load user code; the job is injected at build time.
3. The binary is copied to every candidate node, then started with `nohup` as an HTTP server listening on the selected port.
4. Input is assigned to slots. In local-file mode, the input is cut into chunks of at most 64 MB on newline boundaries. In Common Crawl mode, the orchestrator resolves WET URLs and distributes them round-robin.
5. Each worker runs map locally and writes N intermediate bucket files to its node-local working directory in `/tmp`. The partition function is $\text{FNV32a}(\text{key}) \bmod N$.
6. Each reducer then pulls exactly one bucket from each peer via HTTP, reconstructs the key groups in memory, applies the reducer, and exposes the final key/value lines at `/result`.

7. The orchestrator collects every worker result, merges integer counts locally, sorts the final output, and writes the target file.

Architecturally, the most important decision is that N **is fixed for the whole job**. Since the partition function depends on N , elasticity is handled by replacing a failed *host behind a logical slot* rather than by changing the number of reducers in the middle of the run.

3 Build, execution, and deployment

3.1 Prerequisites

- Go 1.25 or newer on the local machine.
- SSH key-based access to the Telecom Paris lab hosts.
- Network access to `tp.telecom-paris.fr`.
- For the Kafka comparison only: Java 17+ on the remote machines.
- On macOS, SSH multiplexing uses a control socket under `/tmp` to avoid Unix socket path-length limitations.

3.2 How to run the project

MapReduce pipeline

```
./mapreduce.sh -job scripts/jobs/wordcount -input data/simple.txt -output result.txt -n 10 -port 9090
```

or, with Common Crawl input:

```
./mapreduce.sh -job scripts/jobs/wordcount -commoncrawl -crawl CC-MAIN-2026-21 \ -files-limit 4 -output result.txt -n 10 -port 9090
```

Stats collection pipeline

```
./run.sh
```

This command fetches hosts, starts an HTTP server on each selected machine, collects `hostname`, `uptime`, and memory data, and performs cleanup on exit.

The current repository state also adds the following operational material:

- built-in jobs under `scripts/jobs/`: `wordcount`, `langdetect`, `domainpop`, and `doccdensity`;
- a dedicated job-model document at `docs/JOBS.md`;
- a batch runner, `run_all_commoncrawl_jobs.sh`, for executing all built-in jobs against Common Crawl while writing per-job outputs, logs, and timing summaries under `run/commoncrawl_jobs/`.

3.3 Deploy workflow for the remote stats pipeline

The repository contains a second distributed workflow, distinct from the MapReduce runtime, whose purpose is to deploy a simple program to many lab machines, collect command outputs, and clean up deterministically. This path is exposed by `run.sh` and implemented by the four tools `scripts/make_hosts`, `scripts/deploy`, `scripts/collect`, and `scripts/cleanup`.

Step-by-step execution

1. **Host discovery.** `run.sh` calls `scripts/make_hosts` with the configured value of `N`. The tool queries the Telecom Paris availability endpoint and writes the selected host list into `hosts.txt`.
2. **Deployment.** `scripts/deploy` parses `manifest.json`, optionally copies listed files to the configured NFS location, then opens bounded-parallel SSH sessions to the selected hosts and executes every command listed in `commands`. In the current repository state, this phase starts a Python HTTP server on each host and writes its PID to `/tmp/httpserver.pid`.
3. **Collection.** `scripts/collect` reuses the same host list and manifest, reads `collect_commands`, runs those commands over SSH on every host, and writes a host-by-host report to `stats.txt`. In the present configuration, the collected values are `hostname`, `uptime`, and the first two lines of `free -m`.
4. **Cleanup.** `scripts/cleanup` runs the `cleanup_commands` from the manifest on each host and, when relevant, removes deployed files from the configured NFS destination. The shell wrapper installs this cleanup phase in a trap so that teardown is attempted even when earlier phases fail.

Configuration role of `manifest.json`

The deploy workflow is parameterized by `manifest.json`:

- `n` limits how many hosts are used by the deploy, collect, and cleanup tools;
- `files` and `nfs` describe optional files to copy to a shared NFS destination before remote execution;
- `commands` defines what the deploy phase starts remotely;
- `collect_commands` defines what is sampled during collection;
- `cleanup_commands` defines how remote state is removed.

Operational properties

Three implementation details are particularly important for the report:

1. SSH and SCP fan-out are bounded by the shared parallelism helpers in `scripts/internal/remote`, which prevents the client from opening an excessive number of concurrent connections.
2. Remote command batches are executed in one SSH session per host, with command strings joined by `&&`, so a command failure stops the rest of that host's sequence.
3. The deploy workflow is intentionally generic: the current repository starts a Python HTTP server, but the same mechanism can launch any other program that can be described through the manifest.

3.4 Exactly what must be changed for another student login

The MapReduce path is largely login-agnostic because it deploys the worker to `/tmp/mr-worker` on each host and does not embed a username in the wrapper script. The portability-sensitive points are therefore limited and can be stated precisely:

1. **`manifest.json`, field `n`.** If the stats pipeline is used, the number of targeted hosts in `manifest.json` should stay consistent with the shell variable `N` in `run.sh`. In the repository snapshot, `run.sh` sets `N=100` and `manifest.json` also has `"n": 100`.

2. **manifest.json, field nfs.** The generic `deploy/cleanup` tools support copying files to an NFS destination of the form `user@host:/shared/path/`. If a student uses that feature, this field must be changed to their own login and path. In the current repository state it is intentionally empty because the provided stats pipeline does not need NFS.
3. **manifest.json, fields commands, collect_commands, and cleanup_commands.** These commands must match the program the student wants to run remotely. The current values start a Python HTTP server and remove its PID file.
4. **Input/output paths passed to mapreduce.sh.** These are not hardcoded, so another student only changes the command-line arguments: `-job`, `-input` or `-commoncrawl`, `-output`, `-n`, and optionally `-port`.
5. **SSH environment.** The project assumes the student already has passwordless SSH configured for the lab machines. That requirement lives outside the repository, but it is the only truly user-specific prerequisite for the MapReduce path.

The recommended validation procedure with another student account is as follows:

1. clone the repository under the second login;
2. verify SSH access to the lab hosts;
3. run `./mapreduce.sh` on a small local input;
4. run `./run.sh` after checking that `run.sh` and `manifest.json` agree on the host count.

Placeholder to fill later: Insert the actual second-student validation result here: student login used, commands run, whether any user-specific path needed adjustment, and final outcome.

4 Protocol and phase execution

4.1 Worker HTTP API

Endpoint	Method	Role
<code>/health</code>	GET	Readiness and liveness probe.
<code>/data</code>	POST	Receives one raw input payload in local-file mode.
<code>/load</code>	POST	Receives a JSON list of Common Crawl URLs and downloads them locally.
<code>/map</code>	POST	Runs map on the locally stored input and creates pre-partitioned bucket files.
<code>/intermediate</code>	GET	Serves one reducer bucket from the local disk.
<code>/reduce</code>	POST	Pulls one bucket from each peer and runs reduce.
<code>/result</code>	GET	Returns the final key/value lines.
<code>/debug/pprof/*</code>	GET	Optional profiling endpoints when workers are started with <code>-pprof</code> .

4.2 MapReduce phase sequence

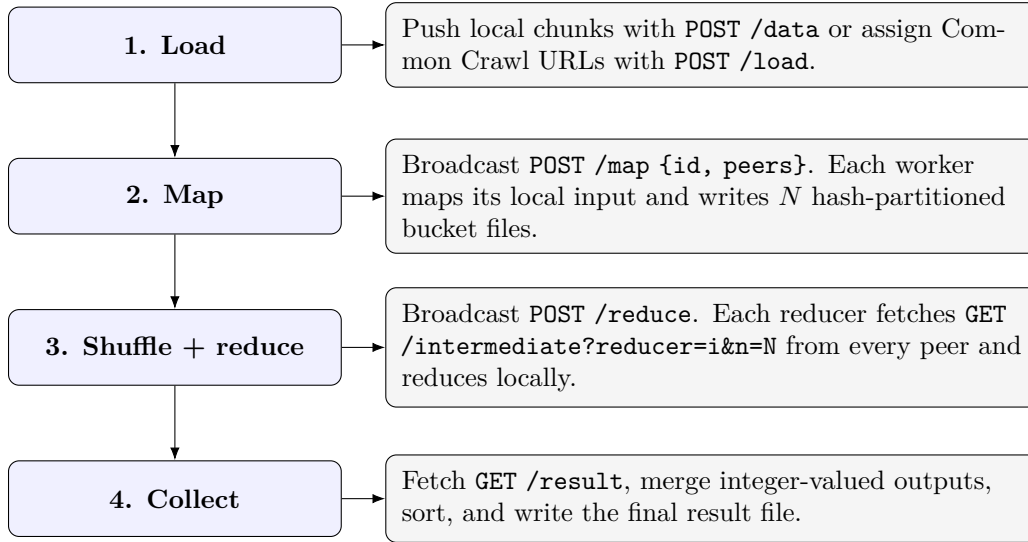


Figure 2: Single-job execution flow. The implementation has one MapReduce stage: load, map, shuffle-by-pull, reduce, then result collection.

Parallelism model: All workers execute a given phase concurrently. The orchestrator advances only after every active slot has reached the required state for that phase.

4.3 Input dispatch depending on dataset size and node count

Local-file mode

For a local input file, the orchestrator reads the whole file and splits it into chunks of at most 64 MB, preferably on newline boundaries. This answers the “splitting” part of the assignment: the split phase exists on the client side before any remote computation starts.

In the current implementation, each slot receives at most one chunk through /data. Concretely, slot i gets chunk i , and if there are more chunks than active slots, the extra chunks are not assigned in the current `run()` path. Therefore:

- if the file produces $\leq N$ chunks, all data is dispatched;
- if the file produces $> N$ chunks, the local-file path is at present incomplete and would need either round-robin multi-chunk assignment or a proper task scheduler.

This behavior is both a design characteristic and a significant remaining limitation.

Common Crawl mode

For Common Crawl, the orchestrator resolves a list of WET URLs and assigns them round-robin across the N slots. Slot i receives URL indices $i, i + N, i + 2N, \dots$. Each worker then downloads its own files directly to its local work directory. This mode does *not* suffer from the one-chunk per slot restriction because a worker may receive many URLs.

If `-files-limit` or `-chunks-limit` is lower than N , some workers may receive no URLs. The repository documentation now explicitly notes this case and recommends increasing the Common Crawl limits or reducing N when a more balanced assignment is required.

Dispatch rationale

The dispatch strategy depends on the input type:

- **Local file:** size-based chunking to keep request sizes bounded and preserve line boundaries.
- **Common Crawl:** URL-level round-robin distribution because the input is already segmented into remote WET files.

Round-robin distribution is straightforward and inexpensive to implement, but it assumes that the remote files are broadly comparable in size. If some WET files are substantially larger than others, the cluster remains susceptible to input skew.

4.4 Timing of each phase

The implementation measures and logs the compute phases after the data has already been loaded onto the workers:

- load or upload is executed first and is excluded from the main `TIMING compute_seconds=...` line;
- map, reduce, and result collection are timed explicitly;
- final local merge is part of result collection from the user point of view, but the code reports the `collect_seconds` window separately from the total compute window.

There is **no second MapReduce stage** in the current implementation. The workflow is a single-stage MapReduce job. If an application requires two consecutive analytical passes, they must presently be executed as two separate jobs.

Phase	Status in current report
Client-side splitting	[fill later: insert measured time]
Input load / upload	[fill later: insert measured time]
Map	[fill later: insert measured time]
Shuffle	[fill later: derive or estimate from logs / profiling]
Reduce	[fill later: insert measured time]
Result collection and merge	[fill later: insert measured time]
Second MapReduce stage	Not applicable in the current design

Table 2: Required phase breakdown. The code already emits timings for map, reduce, collect, and total compute; split and load can be measured around the wrapper or orchestrator.

5 Project status: achieved functionality, unmet objectives, and remaining issues

5.1 Achieved

- A distributed MapReduce framework in Go, deployable on shared lab machines without root privileges.
- Pluggable job logic via a small `Mapper/Reducer` API.
- Support for both local files and Common Crawl WET inputs.

- A pull-based shuffle built on pre-partitioned bucket files.
- Bounded parallel SSH/SCP fan-out for deployment, collection, and cleanup.
- Fault handling through a slot-based coordinator with spare hosts, replacement, epoch tracking, and active health checks.
- Experiment scripts for Amdahl-law benchmarking and deployment-path benchmarking.
- A separate Kafka Streams benchmark path to support a system-level comparison.

5.2 Outstanding items

- Fully general local-file scheduling for datasets larger than the number of available slots times one chunk. The code contains a round-robin multi-chunk helper, but the main execution path presently assigns only one chunk per slot.
- A fully populated empirical section in the report. The repository contains the scripts and plotting support, but the actual measurements remain to be produced and inserted.
- Durable storage or replicated intermediate data. The system is a batch engine, not a persistent data platform.
- Multi-stage job chaining inside one execution graph.

5.3 Validated job executions recorded in the repository

The current repository documentation now includes concrete single-worker protocol validation results for the three analytical jobs added beyond word count. These executions used the actual worker HTTP pipeline locally (`/data` $\rightarrow$ `/map` $\rightarrow$ `/reduce` $\rightarrow$ `/result`), thereby exercising the real map/reduce code paths while excluding only remote SSH/SCP deployment.

Job	Recorded output	Interpretation
langdetect	lang:en 15	All 15 input lines were classified as English, which matches the reference corpus used in the documented validation run.
domainpop	example.com 2, golang.org 1, news.ycombinator.com 2	The reducer preserved the expected domain counts from the synthetic WARC-style input used for validation.
docdensity	hist:len.00000-00079 15, stat:chars.alnum 386, stat:chars.other 73, stat:chars.total 459, stat:lines 15	The derived metrics reported in the repository documentation are <code>avg_line_length</code> = 30.600 and <code>alnum_density</code> = 0.840959, confirming that the counter-based output contract is sufficient for post-processing.

These documented validation results do not replace the distributed benchmark section of this report, but they do confirm that the new job implementations remain consistent with the worker protocol and integer-valued merge contract.

5.4 Remaining problems

1. **Scalability bottleneck in local-file mode:** at present the largest correct local dataset is limited by the one-chunk-per-slot behavior.
2. **Memory pressure in map and shuffle:** workers materialize large in-memory maps and arrays before writing or serving buckets.
3. **Serial final merge:** the orchestrator continues to merge and sort results on one machine.

4. **No data replication:** if a slot fails and no spare host is available, the whole job fails.
5. **Skew sensitivity:** URL round-robin is straightforward but does not account for the true byte size or complexity of each WET file.

6 Maximum dataset size achieved

The empirical maximum dataset size should be reported from real experiments, so the exact number is left as a placeholder below. However, the implementation already reveals two distinct limits:

- **Local-file mode:** the current path is only correct while the number of generated chunks does not exceed the number of active slots. Since the chunk size is 64 MB, the practical bound is approximately $64 \text{ MB} \times N$ before the current scheduler must be improved.
- **Common Crawl mode:** there is no such one-chunk structural limit because workers can receive many URLs. The effective maximum depends on available worker disk space under `/tmp`, network time, and job duration.

Placeholder to fill later: Insert the largest dataset actually processed successfully: dataset identity, total bytes, node count, job used, and whether the run was local-file or Common Crawl based.

7 Fault tolerance and crash behavior

7.1 Slot-based recovery model

The coordinator tracks each logical worker slot through the states `pending` $\rightarrow$ `loaded` $\rightarrow$ `mapped` $\rightarrow$ `reduced` $\rightarrow$ `done`. A slot is not bound permanently to one physical machine. If a host fails, the coordinator can rebind the same slot number to a spare host, replay prerequisite phases, and preserve the same global reducer count N .

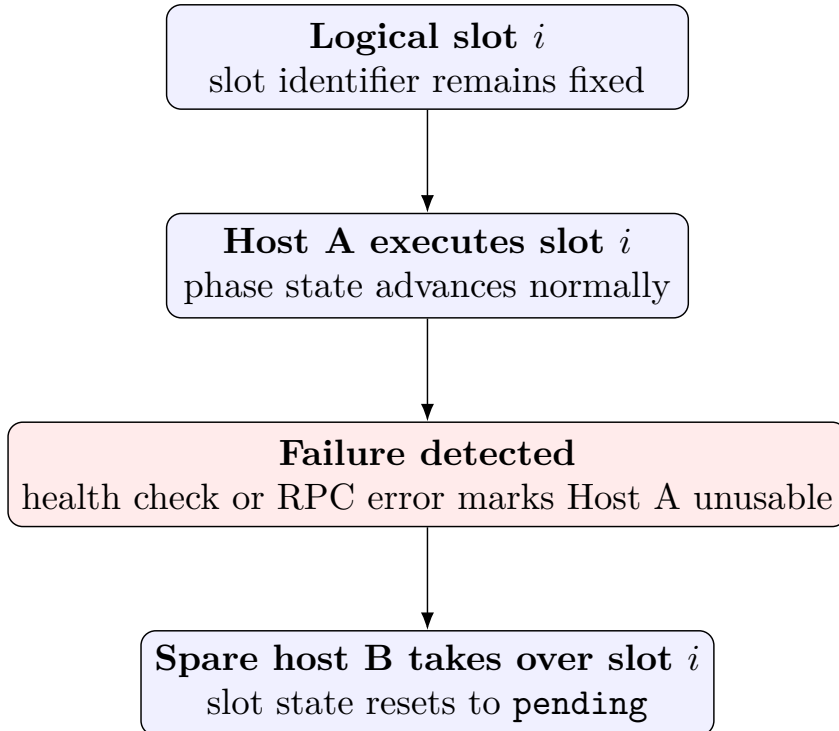


Figure 3: Alternative view of fault handling. The logical slot identifier is preserved, while the physical host can be replaced and the necessary phases are replayed before execution continues.

When a replacement occurs, the new host must replay the prerequisite phases: it reloads the assigned input, reruns map, and only then rejoins the current phase. If failure occurs after reduce has already run, the coordinator bumps the epoch and invalidated reduces are recomputed against the updated peer set.

7.2 Failure scenario: node crash

If a node crashes during the job:

1. the health watcher polls `/health` periodically and marks a host as dead after consecutive failures;
2. transport or HTTP errors during phase execution also identify a culprit slot or failing peer;
3. the coordinator removes that host from future use and claims a spare;
4. the replaced slot is reset to `pending`, so it reloads input and re-runs map before rejoining later phases;
5. if the failure happened after reduce had already been completed by some workers, an epoch counter forces those reduce results to be recomputed because their previous peer set is now stale;
6. if no spare host is available, the job stops with an explicit error.

7.3 Potential resilience improvements

- replicate intermediate buckets to another worker;
- checkpoint map completion and input assignment outside the process so recovery survives orchestrator crashes as well;
- replace in-memory state with a persistent metadata log;
- support multiple spare tiers or automatic host re-discovery;
- stream intermediate data so restart cost is lower for very large jobs.

8 Threads, concurrency, and additional parallelization opportunities

8.1 Current use of threads

In Go, the project uses goroutines rather than manual POSIX threads; however, for the purposes of the assignment, these goroutines constitute the active thread-level concurrency mechanism of the system.

Location	Concurrent work	Reason
scripts/internal/remote	Bounded worker pools	Limit SSH/SCP fan-out while preserving parallel execution across many hosts.
scripts/deploy	One goroutine per host copy/command batch	Reduce deployment latency across many machines.
scripts/collect	Parallel SSH command execution	Reduce collection latency for remote statistics.
scripts/cleanup	Parallel remote cleanup	Reduce shutdown time.
scripts/mapreduce	Parallel deployment, health checks, phase broadcasts, result fetches	Drive all workers concurrently rather than serially.
scripts/mapreduce/coordinator.go	One goroutine per slot in each pass; one health watcher thread	Advance slots independently and detect failures continuously.
scripts/worker	One goroutine per peer during reduce fetch	Pull shuffle data from all peers concurrently.

8.2 Areas where additional threads could be beneficial

- parallelizing map inside one worker across multiple local input files or independent line batches;
- parallelizing Common Crawl downloads per worker;
- streaming and decoding intermediate buckets concurrently instead of decoding each full JSON payload into memory first;
- parallelizing the final merge and sort on the orchestrator side.

9 Storage, communication, and optimization opportunities

9.1 Memory or disk for intermediate results?

The current design uses **both**, but for different stages:

- during map, workers first build in-memory structures (`map[string][]string`) and only then partition them into bucket arrays;
- the partitioned intermediate output is then written to **node-local disk** under the worker's temporary directory;
- during reduce, the fetched intermediate buckets are read back into memory and regrouped by key before the reducer runs.

Accordingly, the authoritative copy of intermediate shuffle data resides on the worker's local disk, not on NFS and not exclusively in RAM. This choice is beneficial for avoiding NFS contention, although the current implementation still uses more peak memory than necessary.

9.2 FTP or sockets?

The project does **not** use FTP.

- host deployment and cleanup use SSH and SCP;

- MapReduce data exchange uses HTTP over TCP sockets;
- Common Crawl downloads also use HTTP;
- the stats pipeline collects command output over SSH.

9.3 Optimization ideas

1. **Streaming map buckets:** write key/value lines directly to the target reducer files instead of materializing all reducer buckets in memory.
2. **Streaming shuffle:** serve the `.jsonl` bucket files as a stream instead of decoding into an array and re-encoding them as JSON.
3. **Combiner support:** for associative jobs such as word count, a local combiner would shrink the shuffle dramatically.
4. **Better local-file scheduling:** use round-robin multi-chunk assignment or a true task queue so large inputs scale beyond N chunks.
5. **Data-aware balancing:** weight Common Crawl files by size rather than by count only.
6. **Parallel final merge:** the orchestrator can merge partial maps in parallel before sorting.

10 Experiments and results placeholders

10.1 Recommended experimental methodology

The repository already contains the required experimental helpers:

- `experiments/run_benchmark.sh` for strong-scaling and Amdahl-law runs;
- `experiments/amdahl.gnuplot` to render the speedup plot;
- `experiments/run_deploy_benchmark.sh` to time deploy/collect/cleanup separately;
- the orchestrator log line `TIMINGnodes=...map_seconds=...reduce_seconds=...collect_seconds=...compute_seconds=...`

The report should use medians over multiple runs because the laboratory machines are shared and therefore subject to performance variability.

10.2 Results for different dataset sizes and node counts

Dataset	Bytes	Nodes	Median map (s)	Median reduce (s)	Median compute (s)
Small local test	[fill later: fill]	1	[fill later: fill]	[fill later: fill]	[fill later: fill]
Small local test	[fill later: fill]	2	[fill later: fill]	[fill later: fill]	[fill later: fill]
Common Crawl set A	[fill later: fill]	4	[fill later: fill]	[fill later: fill]	[fill later: fill]
Common Crawl set B	[fill later: fill]	8	[fill later: fill]	[fill later: fill]	[fill later: fill]

Table 3: Placeholder table for the required experimental results over multiple dataset sizes and machine counts.

Placeholder to fill later: Replace the table above with the real median timings for all dataset sizes and machine counts used in the final evaluation.

10.3 Amdahl's law: how to validate it empirically

Yes, Amdahl's law can be validated empirically with this repository, provided that the workload is kept fixed while the number of nodes changes. The supplied `run_benchmark.sh` script already implements the correct methodology: it keeps one Common Crawl workload fixed and varies N over $\{1, 2, 4, 8, \dots\}$.

What should be the speedup reference?

The correct baseline is:

$$S(N) = \frac{T(1)}{T(N)}$$

where $T(1)$ is the median runtime of the **same MapReduce algorithm on the same dataset** with one active node. In other words:

- the baseline should be the 1-node run, not the 2-node run;
- the algorithm should remain the custom MapReduce pipeline so that only cluster size changes;
- a comparison with a standard sequential algorithm remains useful, but it answers a different question than Amdahl scaling.

What graph shape should be expected?

For a fixed dataset size, the speedup curve should generally:

1. rise close to linearly for small N ;
2. bend once communication, orchestration, and serial merge costs become significant;
3. eventually flatten, because the serial fraction and shuffle overhead limit further gains.

In this project, the main causes of deviation from ideal linear speedup are the serial final merge, HTTP shuffle overhead, and skew across workers.

Should the graph be tried with multiple dataset sizes?

Yes. Running the same node sweep for several dataset sizes is valuable because the curve may change:

- small datasets often saturate quickly because startup and communication dominate;
- larger datasets usually expose a higher effective parallel fraction because computation dominates fixed orchestration overheads longer.

Can the global parallel portion be deduced?

Yes. Once the measured speedup curve is available, the global parallel portion p can be estimated from Amdahl's law:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}$$

By fitting the measured points, one can estimate p and therefore the effective serial fraction $1-p$. **Placeholder to fill later:** Insert the measured speedup plot, fitted Amdahl curve, and the estimated parallel portion p . Also discuss whether the estimate changes with dataset size.

10.4 Interpreting the phase breakdown

The code naturally separates the following costs:

- split and load cost before compute timing begins;
- map cost, which should scale best;
- shuffle cost, partly embedded in reduce because reducers pull from peers;
- reduce cost, which depends on key skew and network traffic;
- collection and final merge cost, which becomes increasingly serial.

The report should therefore comment not only on total runtime, but also on which phase stops scaling first.

11 Comparison with Kafka Streams

11.1 What is comparable

The repository contains a Kafka benchmark path under `kafka/`:

- `deploy_kafka.sh` deploys Kafka brokers in KRaft mode;
- `WordCountJob.java` implements the word-count logic;
- `run_kafka_wordcount.sh` produces input, runs the job, and prints a `TIMINGcompute_seconds=...` line;
- `clean_kafka.sh` removes the cluster state.

The appropriate comparison is to use the **same input file** and the **same number of nodes** for both systems, then compare medians.

11.2 Qualitative comparison

Aspect	Custom Go MapReduce	Kafka Streams
Execution model	Batch, phase-oriented job	Continuous stream processing on top of a persistent log
Persistence	Intermediate data is local and temporary	Records are persisted in Kafka topics
Fault tolerance	Spare-host replacement if available, but no durable replicated buckets	Built-in broker replication and consumer-group recovery
Operational weight	Relatively light: Go binary + SSH + HTTP	More substantial: Java, Kafka brokers, topics, partitioning, cluster management
Latency profile	Incurs per-phase orchestration and shuffle costs	Incurs per-record logging and streaming overhead
Best fit	Batch analytics over temporary data	Durable, reusable, incrementally processed event streams

11.3 Interpretation of the comparison

Kafka is not simply “superior” or “inferior.” It addresses a broader operational problem. It is expected to impose a larger operational footprint because it offers persistent logs, partition management, and stronger recovery semantics. The custom Go framework is lighter to understand and deploy for batch experiments, but it relinquishes durability and a large portion of the ecosystem support that Kafka provides.

Accordingly, the comparison may be summarized as follows:

- the custom system is appropriate for experimentation and batch analytics under tight environment constraints;
- Kafka becomes preferable when the workload requires durable input retention, replay, incremental computation, or production-grade resilience.

System	Nodes	Dataset	Median compute (s)
Go MapReduce	[fill later: fill]	[fill later: fill]	[fill later: fill]
Kafka Streams	[fill later: fill]	[fill later: fill]	[fill later: fill]

Table 4: Placeholder for the required performance comparison with Kafka.

12 Conclusion

The repository already contains a substantial distributed system: host discovery, remote deployment, a pluggable MapReduce runtime, fault-aware slot replacement, experiment scripts, and a Kafka comparison path. The architecture is coherent for a lab setting because it deliberately avoids heavy external dependencies and instead builds on SSH, HTTP, and node-local temporary storage.

The principal technical strengths are the simplicity of deployment, the clear job API, and the slot-based recovery model. The principal weaknesses are the one-chunk-per-slot limitation in local-file mode, memory-heavy map/shuffle implementation, and the lack of durable replicated intermediate state.

Placeholder to fill later: Insert the final headline numbers here: maximum dataset size, best speedup obtained, estimated parallel fraction, and the measured Kafka comparison verdict.

A Reproducibility checklist

A.1 Core commands

```
# Run repository Go tests
cd scripts && go test ./...
```

```
# MapReduce on a local file
./mapreduce.sh -job scripts/jobs/wordcount -input data/simple.txt -output result.txt -n
10
```

```
# MapReduce on Common Crawl
./mapreduce.sh -job scripts/jobs/wordcount -commoncrawl -files-limit 10 -n 10 -output wc.
txt
```

```
# Other provided jobs
./mapreduce.sh -job scripts/jobs/langdetect -commoncrawl -files-limit 10 -n 10 -output
lang.txt
./mapreduce.sh -job scripts/jobs/domainpop -commoncrawl -files-limit 10 -n 10 -output
domains.txt
```



```

./mapreduce.sh -job scripts/jobs/docdensity -commoncrawl -files-limit 10 -n 10 -output
density.txt

# Deployment/stats benchmark
experiments/run_deploy_benchmark.sh -n 100 -parallel-values "4 8 16 32" -reps 3

# Amdahl benchmark
experiments/run_benchmark.sh -job scripts/jobs/wordcount -chunks-limit 256 -reps 3

# Run all built-in jobs on Common Crawl
./run_all_commoncrawl_jobs.sh -crawl CC-MAIN-2026-21 -files-limit 1 -chunks-limit 1 -n 4

# Kafka comparison
(cd kafka && ./deploy_kafka.sh -n 10 && ./run_kafka_wordcount.sh -input /tmp/cc_split.txt
&& ./clean_kafka.sh)

```

A.2 Docdensity post-processing

```

awk -F'\t' ' $1=="stat:chars.total"{t=$2} $1=="stat:chars.alnum">{a=$2} $1=="stat:lines">{l=
$2}
END{printf "avg line len = %.1f chars\nalnum density = %.3f\n", t/l, a/t}'
density.txt

```